

Homework 04

Question 1 Answers

Part a

```
library(glmnet)
```

Loading required package: Matrix

Loaded glmnet 5.0

```
set.seed(43246)
n <- 120
p <- 10
n_sim <- 200
beta <- c(0.15, 0.15, 0.30, 0.15, rep(0, 6))
lambda_grid <- c(0.24, 0.18, 0.12, 0.08)
beta_hat <- array(0, dim = c(n_sim, p, length(lambda_grid)))
converged <- logical(n_sim)

for (s in 1:n_sim) {
  U <- matrix(rnorm(n * p), nrow = n, ncol = p)
  X <- U
  X[, 2] <- 0.999 * U[, 1] + sqrt(1 - 0.999^2) * U[, 2]
  y <- as.vector(X %*% beta) + rnorm(n)

  x_bar <- colMeans(X)
  X_centered <- sweep(X, 2, x_bar, "-")
  s_j <- sqrt(colMeans(X_centered^2))
  X_std <- sweep(X_centered, 2, s_j, "/")
  y_centered <- y - mean(y)

  fit <- glmnet(
    X_std, y_centered,
    alpha = 1, lambda = lambda_grid,
    standardize = FALSE, intercept = FALSE,
    control = list(thresh = 1e-18, maxit = 1000000)
  )
  converged[s] <- fit$jerr == 0
  beta_hat[s, , ] <- sweep(as.matrix(fit$beta), 1, s_j, "/")
}

print(all(converged))
```

```
[1] TRUE
```

```
selected <- abs(beta_hat) > 1e-8
selection_frequency <- apply(selected, c(2, 3), mean)
mean_magnitude <- apply(abs(beta_hat), c(2, 3), mean)
dimnames(selection_frequency) <- list(paste0("X", 1:p), paste0("lambda=", lambda_grid))
dimnames(mean_magnitude) <- dimnames(selection_frequency)
round(selection_frequency, 3)
```

	lambda=0.24	lambda=0.18	lambda=0.12	lambda=0.08
X1	0.335	0.390	0.445	0.480
X2	0.390	0.500	0.530	0.575
X3	0.700	0.910	0.965	0.985
X4	0.150	0.320	0.595	0.735
X5	0.015	0.065	0.165	0.355
X6	0.020	0.060	0.210	0.370
X7	0.005	0.065	0.205	0.350
X8	0.010	0.040	0.185	0.370
X9	0.010	0.105	0.210	0.390
X10	0.010	0.075	0.190	0.325

```
round(mean_magnitude, 3)
```

	lambda=0.24	lambda=0.18	lambda=0.12	lambda=0.08
X1	0.032	0.054	0.079	0.099
X2	0.037	0.063	0.092	0.112
X3	0.075	0.123	0.179	0.218
X4	0.007	0.021	0.048	0.075
X5	0.001	0.003	0.009	0.020
X6	0.001	0.003	0.010	0.022
X7	0.000	0.002	0.010	0.021
X8	0.000	0.001	0.008	0.019
X9	0.000	0.003	0.012	0.024
X10	0.000	0.002	0.009	0.018

```
curve_colors <- c("#C84A16", "#9A5B13", "#2F6FB3", "#13294B")
```

```
par(mfrow = c(1, 2), mar = c(4.2, 4.2, 1.2, 0.8))
```

```
matplot(
```

```
  1:p, selection_frequency, type = "l", lty = 1, lwd = 2, col = curve_colors,
  ylim = c(0, 1), xlab = "Covariate j", ylab = "Selection frequency"
```

```
)
```

```
legend("topright", legend = paste("lambda =", lambda_grid), col = curve_colors, lty = 1, lwd = 2, bty =
```

```
matplot(
```

```
  1:p, mean_magnitude, type = "l", lty = 1, lwd = 2, col = curve_colors,
  xlab = "Covariate j", ylab = expression("Mean " * "|" * hat(beta)[j] * "|")
```

```
)
```

```
legend("topright", legend = paste("lambda =", lambda_grid), col = curve_colors, lty = 1, lwd = 2, bty =
```

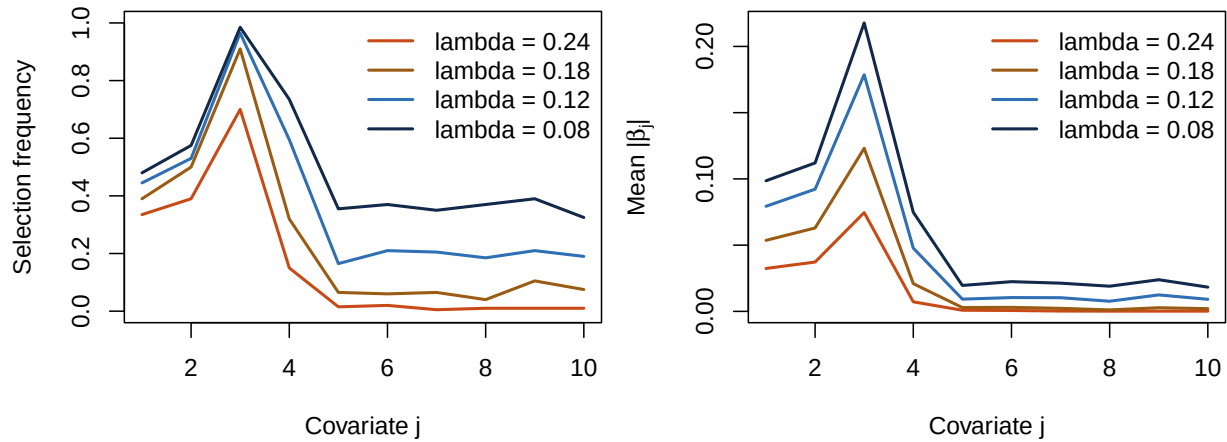


Figure 1: Selection frequency and mean absolute fitted slope by covariate and penalty (lasso).

All 800 fits converge. Increasing the penalty lowers every selection frequency and every mean absolute slope: the noise average ($X_5, \dots, X_{10}$) falls from 36% at $\lambda = 0.08$ to 1.2% at $\lambda = 0.24$, and X_3 's mean magnitude falls from 0.218 to 0.075. Soft-thresholding subtracts λ from each coordinate's score before it can be nonzero, so a larger λ both shrinks surviving slopes and drops more of them to zero.

Part b

```
pair_outcomes <- t(sapply(seq_along(lambda_grid), function(l) {
  sel <- selected[, , l]
  c(
    X1_only = mean(sel[, 1] & !sel[, 2]),
    X2_only = mean(!sel[, 1] & sel[, 2]),
    both = mean(sel[, 1] & sel[, 2]),
    neither = mean(!sel[, 1] & !sel[, 2])
  )
}))
rownames(pair_outcomes) <- paste0("lambda=", lambda_grid)
round(pair_outcomes, 3)
```

	X1_only	X2_only	both	neither
lambda=0.24	0.315	0.370	0.02	0.295
lambda=0.18	0.370	0.480	0.02	0.130
lambda=0.12	0.415	0.500	0.03	0.055
lambda=0.08	0.410	0.505	0.07	0.015

X_1 and X_2 's marginal frequencies (each 0.44–0.58) are close at every penalty because the two are exchangeable: same coefficient, same marginal variance, correlation ≈ 0.999 . As λ decreases, “neither” falls from 30% to 1.5% while “both” stays low (2–7%); most mass sits in the two single-member outcomes. Because $X_1 \approx X_2$, their combined effect is represented almost as well by either alone, and the linear ℓ_1 penalty is indifferent to how a fixed sum splits between collinear coordinates — coordinate descent loads the effect onto whichever score is larger and zeros the other, so the fitted rule can omit one member even though both have nonzero population coefficients.

Part c

```
signal_comparison <- data.frame(
  lambda = lambda_grid,
  X3 = selection_frequency[3, ],
  X4 = selection_frequency[4, ],
  mean_X5_to_X10 = colMeans(selection_frequency[5:10, ])
)
round(signal_comparison, 3)
```

	lambda	X3	X4	mean_X5_to_X10
lambda=0.24	0.24	0.700	0.150	0.012
lambda=0.18	0.18	0.910	0.320	0.068
lambda=0.12	0.12	0.965	0.595	0.194
lambda=0.08	0.08	0.985	0.735	0.360

X_3 (coefficient 0.30) is selected far more often than X_4 (coefficient 0.15) at every penalty, and both exceed the noise average: at $\lambda = 0.12$, $X_3 = 0.965$, $X_4 = 0.595$, noise average = 0.194. An independent covariate's score concentrates near its true coefficient, so a larger coefficient more reliably clears the λ threshold. The pair shows why this can mislead: X_1, X_2 each have true coefficient 0.15 — same as X_4 — yet each is omitted about half the time at $\lambda = 0.12$, so a fitted zero does not mean the population coefficient is zero. And whichever member is kept is standing in for information shared with the one that was dropped, so a nonzero fitted slope does not mean that variable alone matters.

Question 2 Answers

Part a

```
set.seed(43246)
beta_hat2 <- array(0, dim = c(n_sim, p, length(lambda_grid)))
converged2 <- logical(n_sim)

for (s in 1:n_sim) {
  U <- matrix(rnorm(n * p), nrow = n, ncol = p)
  X <- U
  X[, 2] <- 0.999 * U[, 1] + sqrt(1 - 0.999^2) * U[, 2]
  y <- as.vector(X %*% beta) + rnorm(n)

  x_bar <- colMeans(X)
  X_centered <- sweep(X, 2, x_bar, "-")
  s_j <- sqrt(colMeans(X_centered^2))
  X_std <- sweep(X_centered, 2, s_j, "/")
  y_centered <- y - mean(y)

  fit <- glmnet(
    X_std, y_centered,
    family = gaussian(), alpha = 0.5, lambda = lambda_grid,
    standardize = FALSE, intercept = FALSE,
    control = list(thresh = 1e-18, maxit = 1000000)
  )
  converged2[s] <- fit$jerr == 0
  beta_hat2[s, , ] <- sweep(as.matrix(fit$beta), 1, s_j, "/")
}
```

```
print(all(converged2))
```

```
[1] TRUE
```

```
selected2 <- abs(beta_hat2) > 1e-8
selection_frequency2 <- apply(selected2, c(2, 3), mean)
dimnames(selection_frequency2) <- dimnames(selection_frequency)
round(selection_frequency2, 3)
```

	lambda=0.24	lambda=0.18	lambda=0.12	lambda=0.08
X1	0.910	0.940	0.950	0.950
X2	0.935	0.975	0.975	0.980
X3	0.965	0.985	0.985	1.000
X4	0.600	0.720	0.800	0.875
X5	0.175	0.310	0.475	0.635
X6	0.205	0.340	0.465	0.610
X7	0.215	0.310	0.450	0.590
X8	0.190	0.345	0.505	0.660
X9	0.210	0.350	0.495	0.630
X10	0.200	0.270	0.455	0.620

```
par(mar = c(4.2, 4.2, 1.2, 0.8))
matplot(
  1:p, selection_frequency2, type = "l", lty = 1, lwd = 2, col = curve_colors,
  ylim = c(0, 1), xlab = "Covariate j", ylab = "Selection frequency"
)
legend("topright", legend = paste("lambda =", lambda_grid), col = curve_colors, lty = 1, lwd = 2, bty =
```

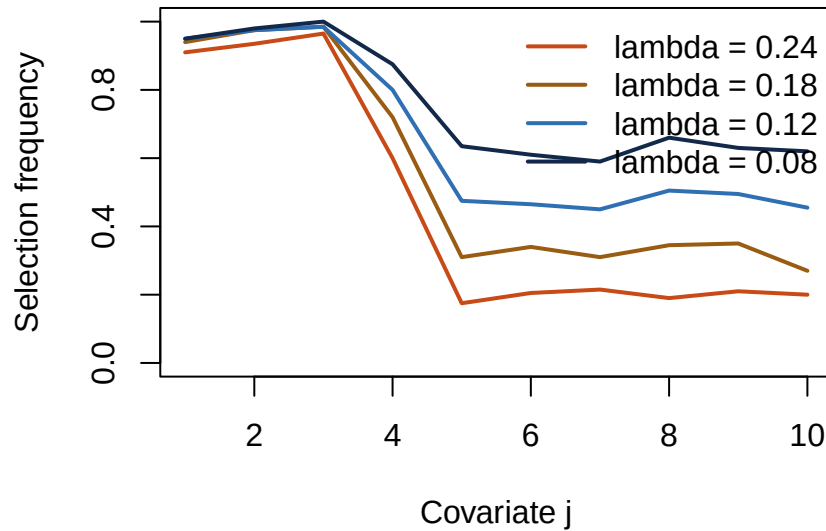


Figure 2: Elastic net selection frequency by covariate and penalty.

Every covariate is selected more often under the elastic net than the lasso at the same λ , and the correlated pair changes the most. At $\lambda = 0.12$: X_1 rises from 0.445 to 0.950 and X_2 from 0.530 to 0.975 (gains of 50.5 and 44.5 points), versus a 28.0-point gain for the noise average (0.194 $\rightarrow$ 0.474) and a smaller gain for X_4 (0.595 $\rightarrow$ 0.800).

Part b

```
pair_outcomes2 <- t(apply(seq_along(lambda_grid), function(l) {
  sel <- selected2[, , l]
  c(
    X1_only = mean(sel[, 1] & !sel[, 2]),
    X2_only = mean(!sel[, 1] & sel[, 2]),
    both = mean(sel[, 1] & sel[, 2]),
    neither = mean(!sel[, 1] & !sel[, 2])
  )
}))
rownames(pair_outcomes2) <- paste0("lambda=", lambda_grid)
round(pair_outcomes2, 3)
```

	X1_only	X2_only	both	neither
lambda=0.24	0.005	0.03	0.905	0.06
lambda=0.18	0.005	0.04	0.935	0.02
lambda=0.12	0.015	0.04	0.935	0.01
lambda=0.08	0.020	0.05	0.930	0.00

“Both” now dominates at every penalty (0.905–0.935), versus 2–7% under the lasso, so the pair is properly selected together in most repetitions. The squared ℓ_2 term explains why: splitting a fixed combined effect barely changes the fit or the ℓ_1 cost when $X_1 \approx X_2$, but $\hat{\beta}_1^2 + \hat{\beta}_2^2$ at a fixed sum is minimized by an even split, so this strictly convex term breaks the lasso’s indifference and rewards keeping both.

Part c

```
noise_comparison <- data.frame(
  lambda = lambda_grid,
  lasso_noise = colMeans(selection_frequency[5:10, ]),
  enet_noise = colMeans(selection_frequency2[5:10, ])
)
round(noise_comparison, 3)
```

	lambda	lasso_noise	enet_noise
lambda=0.24	0.24	0.012	0.199
lambda=0.18	0.18	0.068	0.321
lambda=0.12	0.12	0.194	0.474
lambda=0.08	0.08	0.360	0.624

The noise average still falls as λ grows, but far less steeply: $0.624 \rightarrow 0.199$ over the grid, versus $0.360 \rightarrow 0.012$ for the lasso. At equal λ , a standardized coordinate’s threshold under $\alpha = 0.5$ is $\lambda/2$ rather than λ , so the elastic net needs roughly twice the penalty to screen as aggressively. On this grid, $\lambda = 0.24$ best balances both goals: “both” is selected 90.5% of the time while the noise average is at its grid minimum, 19.9%.

Question 3 Answers

Part a

```
lasso_path <- function(X, y, lambda_grid, tol = 1e-8, max_iter = 1000) {
  y <- y - mean(y)
  beta <- rep(0, ncol(X))
  beta_path <- matrix(0, nrow = length(lambda_grid), ncol = ncol(X))
  cycles <- integer(length(lambda_grid))
```

```

for (index in seq_along(lambda_grid)) {
  lambda <- lambda_grid[index]
  for (iteration in seq_len(max_iter)) {
    beta_old <- beta
    for (j in seq_len(ncol(X))) {
      residual_j <- y - drop(X %*% beta) + X[, j] * beta[j]
      a_j <- mean(X[, j] * residual_j)
      beta[j] <- sign(a_j) * max(abs(a_j) - lambda, 0)
    }
    if (max(abs(beta - beta_old)) < tol) break
  }
  if (iteration == max_iter) stop("no convergence at lambda = ", lambda)
  beta_path[index, ] <- beta
  cycles[index] <- iteration
}
list(path = beta_path, cycles = cycles)
}

set.seed(43247)
n3 <- 100
X3 <- matrix(rnorm(n3 * 3), nrow = n3)
y3 <- 1.5 + X3[, 1] - 2 * X3[, 2] + rnorm(n3)

x_bar3 <- colMeans(X3)
s3 <- sqrt(colMeans(sweep(X3, 2, x_bar3, "-")^2))
X3_std <- sweep(sweep(X3, 2, x_bar3, "-"), 2, s3, "/")

lambda_grid3 <- c(2.5, 2.0, 1.5, 1.0, 0.6, 0.4, 0.25, 0.15, 0.08, 0.03)
fit3 <- lasso_path(X3_std, y3, lambda_grid3)
path_table <- data.frame(lambda = lambda_grid3, fit3$path, cycles = fit3$cycles)
names(path_table)[2:4] <- c("beta_1", "beta_2", "beta_3")
round(path_table, 4)

```

	lambda	beta_1	beta_2	beta_3	cycles
1	2.50	0.0000	0.0000	0	1
2	2.00	0.0000	0.0000	0	1
3	1.50	0.0000	-0.3272	0	2
4	1.00	0.0000	-0.8272	0	2
5	0.60	0.2835	-1.2495	0	5
6	0.40	0.5006	-1.4666	0	5
7	0.25	0.6634	-1.6294	0	5
8	0.15	0.7719	-1.7380	0	5
9	0.08	0.8479	-1.8140	0	5
10	0.03	0.9022	-1.8682	0	5

```

y3_centered <- y3 - mean(y3)
lambda_max <- max(abs(colMeans(X3_std * y3_centered)))
print(lambda_max)

```

```
[1] 1.827201
```

```
print(coef(lm(y3_centered ~ X3_std - 1)))
```

	X3_std1	X3_std2	X3_std3
	0.932892617	-1.901630629	0.008847341

Every fit converges (the function would stop otherwise). $\lambda_{\max} = 1.827$, and every slope is exactly zero at the two grid values at or above it (2.5, 2.0), as required.

Part b

```
order3 <- order(log(lambda_grid3))
path_colors <- c("#2F6FB3", "#C84A16", "#13294B")
par(mar = c(4.2, 4.2, 1.2, 0.8))
matplot(
  log(lambda_grid3[order3]), fit3$path[order3, ],
  type = "l", lty = 1, lwd = 2, col = path_colors,
  xlab = "log(lambda)", ylab = "Fitted slope"
)
abline(h = 0, col = "grey80")
legend("bottomleft", legend = c("X1", "X2", "X3"), col = path_colors, lty = 1, lwd = 2, bty = "n")
```

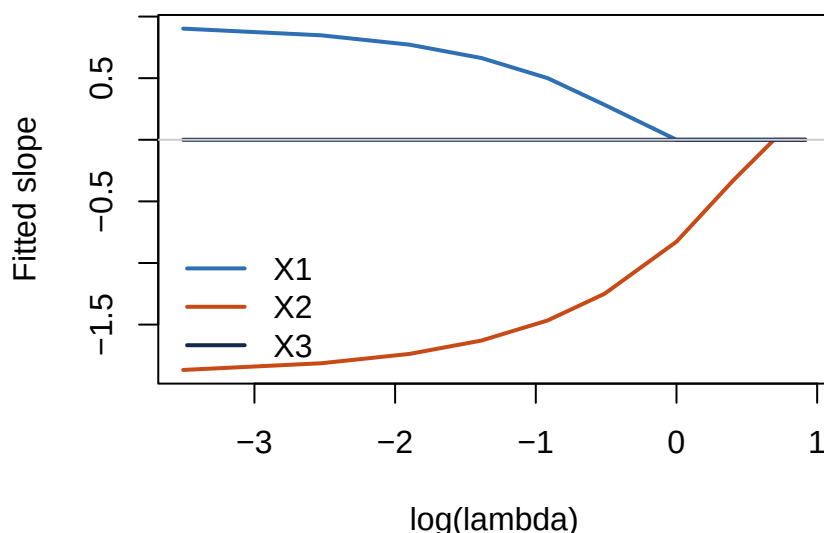


Figure 3: Lasso coefficient path from coordinate descent.

X_2 enters first (nonzero by $\lambda = 1.5$), then X_1 (by $\lambda = 0.6$); X_3 never enters. This matches the true coefficients $-2, 1, 0$: a larger true coefficient gives a larger score, which clears the λ threshold at a larger penalty, so covariates enter in decreasing order of true effect size.

Part c

X_3 's population coefficient is zero, and in this run it stays exactly zero at every grid value, including $\lambda = 0.03$. The fit there, $(0.902, -1.868, 0)$, is close to but not identical to the least-squares fit, $(0.933, -1.902, 0.009)$: coordinate descent still thresholds every score by $\lambda = 0.03$, which shrinks the active slopes slightly and keeps X_3 's small least-squares slope inside the threshold band. Only as $\lambda \rightarrow 0$ does the penalized fit converge to the least-squares solution.

Question 4 Answers

```

diabetes <- read.csv("data/diabetes.csv")
split_folds <- read.csv("data/diabetes-split-folds.csv")
diabetes$row_id <- seq_len(nrow(diabetes))
course_data <- merge(split_folds, diabetes, by = "row_id")

features <- c("age", "sex", "bmi", "bp", "s1", "s2", "s3", "s4", "s5", "s6")
is_train <- course_data$split == "train"
X_train <- as.matrix(course_data[is_train, features])
y_train <- course_data$y[is_train]
X_test <- as.matrix(course_data[!is_train, features])
y_test <- course_data$y[!is_train]
fold <- course_data$cv_fold[is_train]

fit_lasso <- cv.glmnet(X_train, y_train, alpha = 1, foldid = fold, standardize = TRUE)
fit_ridge <- cv.glmnet(X_train, y_train, alpha = 0, foldid = fold, standardize = TRUE)
fit_enet <- cv.glmnet(X_train, y_train, alpha = 0.5, foldid = fold, standardize = TRUE)

fits <- list(Lasso = fit_lasso, Ridge = fit_ridge, `Elastic net` = fit_enet)
summary_table <- data.frame(
  method = names(fits),
  lambda_min = sapply(fits, function(f) f$lambda.min),
  cv_mse = sapply(fits, function(f) min(f$cvm))
)
round_summary <- summary_table
round_summary[, 2:3] <- round(round_summary[, 2:3], 3)
round_summary

```

	method	lambda_min	cv_mse
Lasso	Lasso	0.099	3068.424
Ridge	Ridge	4.616	3080.365
Elastic net	Elastic net	0.165	3069.013

```

par(mfrow = c(1, 3), mar = c(4.2, 4.2, 2, 0.8))
plot(fit_lasso); title("Lasso", line = 2.5)
plot(fit_ridge); title("Ridge", line = 2.5)
plot(fit_enet); title("Elastic net", line = 2.5)

```

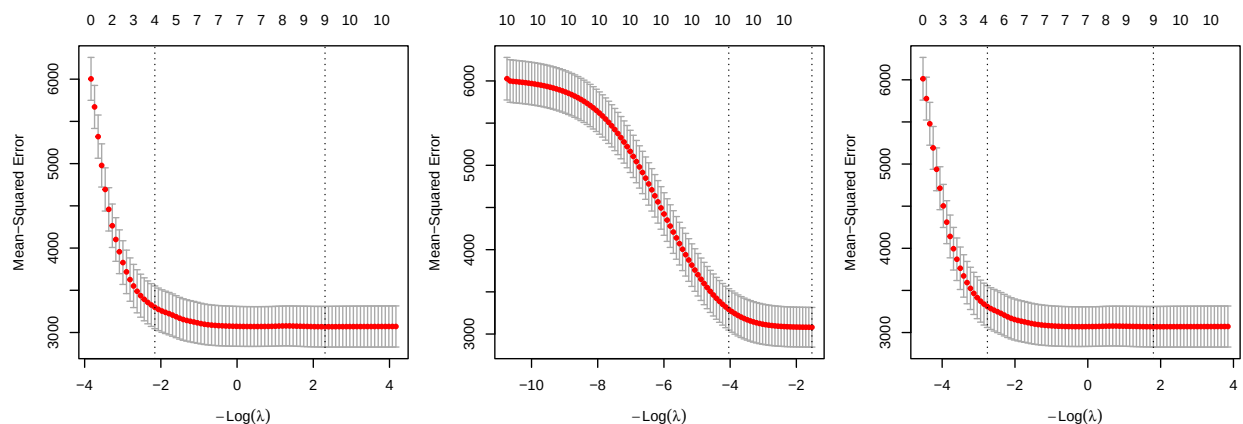


Figure 4: Cross-validation plots for the lasso, ridge, and elastic net fits.

```

winner <- names(fits)[which.min(summary_table$cv_mse)]
best_fit <- fits[[winner]]
beta_hat4 <- coef(best_fit, s = "lambda.min")[-1, 1]
pred_test <- predict(best_fit, newx = X_test, s = "lambda.min")

result <- data.frame(
  method = winner,
  lambda = best_fit$lambda.min,
  nonzero = sum(abs(beta_hat4) > 1e-8),
  cv_mse = min(summary_table$cv_mse),
  test_mse = mean((y_test - pred_test)^2)
)
round(result[, -1], 3)

  lambda nonzero   cv_mse test_mse
1  0.099         9 3068.424 2731.848

print(winner)

```

```
[1] "Lasso"
```

The three tuned fits are nearly indistinguishable on cross-validation error: lasso 3068.4, ridge 3080.4, elastic net 3069.0. Lasso has the smallest minimum and is selected, with $\lambda_{\min} \approx 0.099$, keeping 9 of the 10 covariates (dropping `s3`). Refit on all training rows and evaluated once on the held-out test set, its test MSE is 2731.8, close to its cross-validation error — an honest final evaluation, since the test rows were untouched until the method and penalty were already fixed.